

Taller: CampusTasks

entorno, actualización y eliminación de tareas

Karen Sofía López Botero

2459519-3743

Juan Esteban Pérez Ramírez

2459486

Universidad del Valle

Ingeniería de Sistemas

Desarrollo de Software II

Septiembre-Octubre de 2026

1. Método actualizar en el servicio y su prueba unitaria.

Archivo: backend/src/tareas/tareas.service.ts

```
async actualizar(id: number, titulo: string): Promise<Tarea> {  
  const resultado = await this.db.query<Tarea>(  
    'UPDATE tareas SET titulo = $1 WHERE id = $2 RETURNING id, titulo',  
    [titulo, id],  
  );  
  return resultado.rows[0];  
}
```

Archivo: backend/src/tareas/tareas.service.spec.ts

Prueba unitaria

```
it('actualiza el título y devuelve la fila actualizada', async () => {  
  const actualizada = { id: 1, titulo: 'Tarea actualizada' };  
  query.mockResolvedValue({ rows: [actualizada] });  
  
  await expect(service.actualizar(1, 'Tarea actualizada')).resolves.toEqual(actualizada);  
  expect(query).toHaveBeenCalledWith(  
    'UPDATE tareas SET titulo = $1 WHERE id = $2 RETURNING id, titulo',  
    ['Tarea actualizada', 1],  
  );  
});
```

2. Ruta PATCH /tareas/:id en el controlador y sus pruebas de integración (200 y 404).

Agregamos al: **import { Body, Controller, Get, NotFoundException, Param, Patch, Post } from '@nestjs/common';**

```
@Patch('/:id')  
async actualizar(  
  @Param('id') id: string,  
  @Body('titulo') titulo: string,  
): Promise<Tarea> {  
  const tarea = await this.tareasService.actualizar(Number(id), titulo);  
  if (!tarea) {  
    throw new NotFoundException();  
  }  
  return tarea;  
}
```

- **@Param('id') id: string** Inicialmente es texto (string), lo convierto a número antes de enviarlo para evitar errores y si el servicio no encontró una tarea con ese id, respondemos 404 Not Found.

Prueba integración

```
it('PATCH /tareas/:id actualiza una tarea existente', async () => {
  query.mockResolvedValue({
    rows: [{ id: 1, titulo: 'Título actualizado' }],
  });

  await request(app.getHttpServer())
    .patch('/tareas/1')
    .send({ titulo: 'Título actualizado' })
    .expect(200)
    .expect({ id: 1, titulo: 'Título actualizado' });

  expect(query).toHaveBeenCalledWith(
    'UPDATE tareas SET titulo = $1 WHERE id = $2 RETURNING id, titulo',
    ['Título actualizado', 1],
  );
});

it('PATCH /tareas/:id responde 404 si la tarea no existe', async () => {
  query.mockResolvedValue({
    rows: [],
  });

  await request(app.getHttpServer())
    .patch('/tareas/999')
    .send({ titulo: 'Título actualizado' })
    .expect(404);
});
```

3. Método eliminar en el servicio y su prueba unitaria.

Archivo: backend/src/tareas/tareas.service.ts

```
async eliminar(id: number): Promise<Tarea> {
  const resultado = await this.db.query<Tarea>(
    'DELETE FROM tareas WHERE id = $1 RETURNING id, titulo',
    [id],
  );
  return resultado.rows[0];
}
```

Archivo: backend/src/tareas/tareas.service.spec.ts

Prueba unitaria

```
it('elimina la tarea y devuelve la fila eliminada', async () => {
  const eliminada = { id: 1, titulo: 'Tarea para eliminar' };
  query.mockResolvedValue({ rows: [eliminada] });

  await expect(service.eliminar(1)).resolves.toEqual(eliminada);
  expect(query).toHaveBeenCalledWith(
    'DELETE FROM tareas WHERE id = $1 RETURNING id, titulo',
    [1],
  );
});
```

4. Ruta DELETE /tareas/:id en el controlador y sus pruebas de integración (200 y 404).

```
@Delete('/:id')
async eliminar(@Param('id') id: string) {
  const tarea = await this.tareasService.eliminar(Number(id));

  if (!tarea) {
    throw new NotFoundException();
  }

  return tarea;
}
```

Prueba integración

```
it('DELETE /tareas/:id elimina una tarea existente', async () => {
  query.mockResolvedValue({
    rows: [{ id: 1, titulo: 'Tarea para eliminar' }],
  });

  await request(app.getHttpServer())
    .delete('/tareas/1')
    .expect(200)
    .expect(({ id: 1, titulo: 'Tarea para eliminar' }));

  expect(query).toHaveBeenCalledWith(
    'DELETE FROM tareas WHERE id = $1 RETURNING id, titulo',
    [1],
  );
});

it('DELETE /tareas/:id responde 404 si la tarea no existe', async () => {
  query.mockResolvedValue({
    rows: [],
  });

  await request(app.getHttpServer())
    .delete('/tareas/999')
    .expect(404);
});
```

5. Prueba manual del backend con su base real (por ejemplo con curl o Postman).

```
PS C:\Users\Karen Sofia\Desktop\Dessarrollo 2\campus-task\backen
d> npm run test:e2e

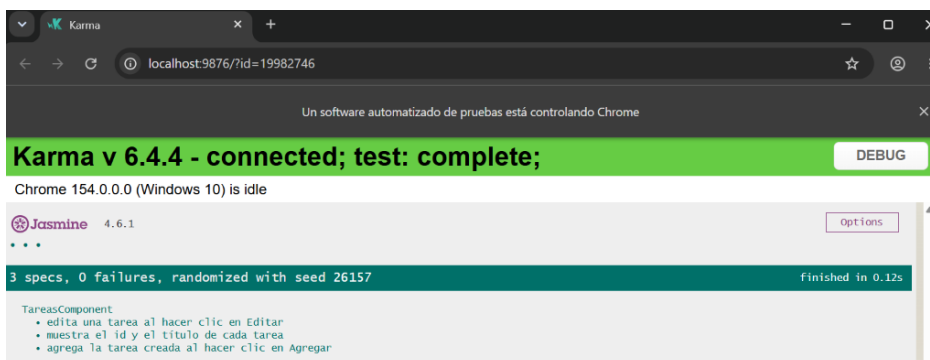
> backend@0.0.1 test:e2e
> jest --config ./test/jest-e2e.json

PASS test/app.e2e-spec.ts
  TareasController (e2e)
    ✓ /tareas (GET) (503 ms)

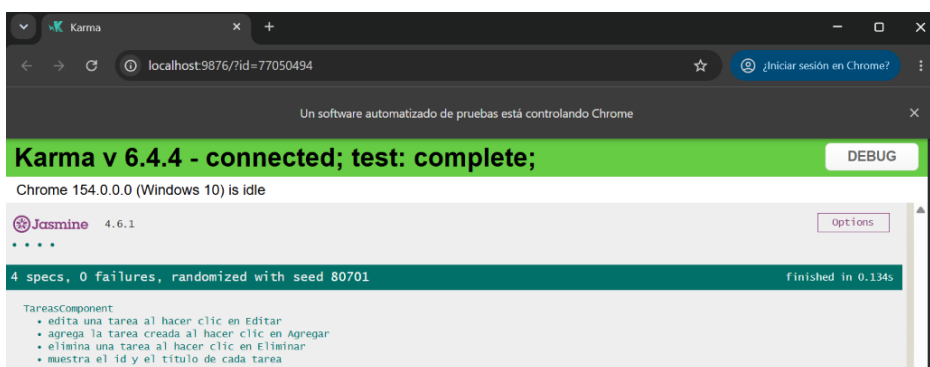
Test Suites: 1 passed, 1 total
Tests: 1 passed, 1 total
Snapshots: 0 total
Time: 1.854 s, estimated 2 s
Ran all test suites.
```

```
StatusCode      : 200
StatusDescription : OK
Content          : [{"id":1,"titulo":"Leer la guía de la
                   clase 2"}, {"id":2,"titulo":"Preparar el
                   entorno de desarrollo"}]
```

6. En el frontend: botón Editar en cada tarea y su prueba de componente.

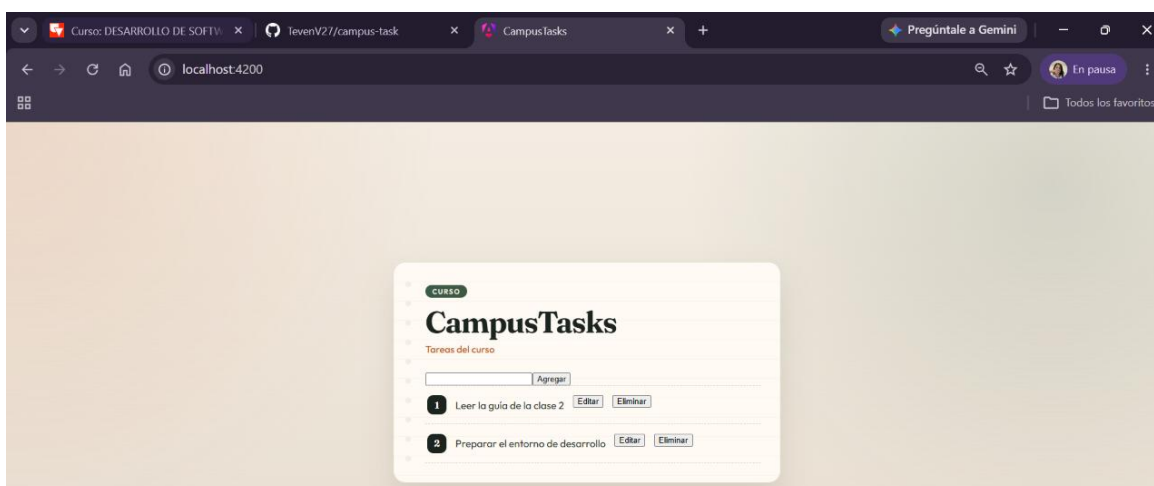


7. En el frontend: botón Eliminar en cada tarea y su prueba de componente.



8. Prueba manual completa desde <http://localhost:4200>: editar y eliminar una tarea y comprobar el cambio en GET /tareass.

Antes de:



Después de:

